

RETAILMART V3 • SQL

DDL & Data Types

WhatsApp Announcement

DDL & DATA TYPES: Build Your First Tables!

Yesterday you explored RetailMart V3 — 55 tables, 2.7 million rows. You looked at the data. Today, you learn how those tables were BUILT. You go from spectator to architect.

Today's Focus:

- CREATE DATABASE, CREATE SCHEMA, CREATE TABLE — building from scratch
- Data Types — INT, SERIAL, VARCHAR, TEXT, NUMERIC, DATE, TIMESTAMP, BOOLEAN
- ALTER TABLE — modify existing structures on the fly
- DROP — destroy structures (carefully!)
- Choosing the RIGHT type for every column — the skill that separates juniors from seniors

Important: ALL practice today happens in YOUR practice database (accio_26), NOT in RetailMart. RetailMart is read-only — you observe its design but build your own tables.

Course Portal: <https://starlordali4444.github.io/postgresql/>

Today you write your FIRST real SQL. Let's go!

— Siraj Sir

#Day3 #DDL #DataTypes #PostgreSQL

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap	5 mins	5 SQL categories, Database hierarchy, pgAdmin connection check
Topic 1: DDL — Building Structures	30 mins	CREATE DATABASE / SCHEMA / TABLE, DROP, IF NOT EXISTS, Naming conventions
Practice Block 1	20 mins	4 problems: Build tables from scratch
Topic 2: Data Types — The Blueprint	35 mins	Numeric types (INT, SERIAL, NUMERIC), Text types (VARCHAR, TEXT, CHAR), Date/Time types, Boolean, JSONB, UUID, The Type Decision Framework
Practice Block 2	20 mins	4 problems: Pick correct types, find & fix wrong types
Topic 3: ALTER TABLE — Modify & Evolve	15 mins	ADD / DROP / RENAME COLUMN, ALTER TYPE, SET DEFAULT, Real migration patterns
Practice Block 3 + Wrap-up	15 mins	4 problems: ALTER TABLE real scenarios, Self-check, homework, Constraints & DML preview

5-Minute Memory Check

From SQL Intro:

- 5 SQL Categories: DDL, DML, DQL, DCL, TCL
- Database Hierarchy: Server → Database → Schema → Table → Row → Cell
- SQL is DECLARATIVE – you say WHAT, engine decides HOW
- Primary Key = unique ID per row. Foreign Key = link between tables.

From Setup & Onboarding:

- PostgreSQL = server process. pgAdmin = GUI client. psql = terminal client.
- RetailMart V3 loaded – 16 schemas, 55 tables, ~2.7M rows
- You ran `SELECT count(*)` and `SELECT * ... LIMIT 5`
- You browsed schemas: customers, sales, products, stores, finance...

Today's Big Question:

'Those 55 tables did not appear by magic.

Someone wrote `CREATE TABLE` 55 times.

Someone chose `INT` vs `VARCHAR` vs `NUMERIC` for every single column.

Today, YOU learn how – and more importantly, WHY.'

The Architect vs The Builder vs The Resident (1/2)

Three people work on a building:

The Architect (DDL):

Draws the blueprint. How many floors? How many rooms per floor? What size is the kitchen? Where do the pipes run? The architect never moves furniture or paints walls. She only defines STRUCTURE.

The Builder (DML):

Follows the blueprint and puts things INSIDE — lays tiles, installs fixtures, moves furniture in. The builder can add furniture (INSERT), rearrange it (UPDATE), or remove it (DELETE). But the builder cannot add a new room — that requires the architect.

The Architect vs The Builder vs The Resident (2/2)

The Resident (DQL):

Lives in the house. Opens doors, looks through windows, checks what is in each room. The resident does not change the structure OR move furniture. She just OBSERVES. That is SELECT.

Today, you become the Architect.

[Mapping to SQL]

Architect draws blueprint	→ CREATE TABLE (define structure)
Architect modifies blueprint	→ ALTER TABLE (change structure)
Architect demolishes a room	→ DROP TABLE (destroy structure)
Builder brings furniture	→ INSERT (add data) – Constraints & DML
Builder rearranges furniture	→ UPDATE (change data) – Constraints & DML
Builder removes furniture	→ DELETE (remove data) – Constraints & DML
Resident looks around	→ SELECT (read data) – Filtering & Sorting+

CREATE DATABASE — The Empty Plot of Land

A database is the top-level container. Everything lives inside it — schemas, tables, views, indexes, functions.

[Syntax & Examples]

-- Basic syntax

```
CREATE DATABASE database_name;
```

-- Create your practice database

```
CREATE DATABASE accio_26;
```

-- What just happened under the hood?

-- 1. PostgreSQL created a NEW directory on disk

-- 2. Copied the template database (template1) as the skeleton

-- 3. Created one default schema inside: 'public'

-- 4. Created system catalogs (pg_catalog, information_schema)

-- 5. The database has ZERO user tables. You build those.

-- Connect to it:

```
\c accio_26    -- in psql
```

-- or: right-click in pgAdmin → Query Tool

CREATE SCHEMA — Folders Inside the Database (1/4)

A schema is a namespace — a logical folder that groups related tables. Without schemas, all 55 tables dump into one flat 'public' schema. Imagine a folder with 55 files and no subfolders — chaos.

[Syntax & Examples]

```
CREATE SCHEMA schema_name;
```

```
-- Organize like a real company
```

```
CREATE SCHEMA shop;           -- products, inventory, suppliers
CREATE SCHEMA people;         -- customers, employees
CREATE SCHEMA transactions;    -- orders, payments, returns
CREATE SCHEMA analytics;       -- views, reports, aggregates
```

CREATE SCHEMA — Folders Inside the Database (2/4)

```
-- Why schemas matter:  
-- 1. Organization: 55 tables in 16 folders vs 55 files loose  
-- 2. Namespace:    shop.products ≠ analytics.products  
-- 3. Permissions:  GRANT SELECT ON ALL TABLES IN SCHEMA finance TO analyst;  
-- 4. Clarity:      SELECT * FROM sales.orders – you KNOW where it lives
```

CREATE SCHEMA — Folders Inside the Database (3/4)

[RetailMart V3's 16 Schemas — Why Each Exists]

Schema	Tables	Purpose
core	6	Static dimensions (dates, regions, categories)
products	4	What we sell (items, suppliers, inventory)
stores	3	Where we sell (locations, staff, expenses)
customers	5	Who buys (profiles, addresses, reviews, wallets)
sales	5	The transactions (orders, items, payments, shipments)
finance	6	Money tracking (accounts, transfers, revenue)
hr	2	People ops (attendance, salary history)
marketing	3	Campaign performance (ads, email clicks)
support	1	Customer complaints (tickets)
web_events	2	Digital tracking (page views, clicks)
supply_chain	3	Logistics (warehouses, shipments, stock levels)
loyalty	3	Rewards program (tiers, members, redemptions)
manufacture	2	Production (lines, work orders)
payroll	2	Salary processing (tax brackets, pay slips)
call_center	2	Phone support (calls, transcripts)
audit	6	System logs (API requests, changes, errors)

CREATE SCHEMA — Folders Inside the Database (4/4)

Total: 16 schemas, 55 tables. Every table has a HOME.

CREATE TABLE — The Main Event (1/5)

This is the DDL command you will use most. It defines a table — its name, columns, and the data type of each column. Every column **MUST** have a type. No exceptions.

[Syntax — The Template]

```
CREATE TABLE schema_name.table_name (  
    column_1 DATA_TYPE,  
    column_2 DATA_TYPE,  
    column_3 DATA_TYPE DEFAULT default_value,  
    ...  
);
```

```
-- If you skip schema_name, it goes into 'public' schema.  
-- ALWAYS specify the schema explicitly. It is a professional habit.
```

CREATE TABLE — The Main Event (2/5)

[Example 1 – A Students Table]

```
CREATE TABLE public.students (  
    student_id    SERIAL,           -- auto: 1, 2, 3, 4...  
    first_name    VARCHAR(50),      -- text, max 50 chars  
    last_name     VARCHAR(50),  
    email         VARCHAR(150),     -- emails can be long  
    age          SMALLINT,          -- small whole number  
    is_active     BOOLEAN DEFAULT true, -- defaults to true  
    enrolled_on   DATE              -- calendar date  
);
```

CREATE TABLE — The Main Event (3/5)

```
-- What happened?  
-- PostgreSQL created a table with 7 columns.  
-- Each column has a NAME and a TYPE.  
-- student_id auto-increments (you never insert it manually).  
-- is_active defaults to true if you skip it during INSERT.  
-- The table has 0 rows. It is an empty blueprint waiting for data.
```

CREATE TABLE — The Main Event (4/5)

[Example 2 — A Products Table (inspired by RetailMart)]

```
CREATE TABLE shop.products (  
    product_id      SERIAL,                -- auto ID  
    product_name    VARCHAR(200),          -- product title  
    brand_name      VARCHAR(100),          -- brand  
    price           NUMERIC(10, 2),        -- up to ₹99,99,999.99  
    cost_price      NUMERIC(10, 2),        -- what WE pay the supplier  
    category        VARCHAR(50),           -- Electronics, Fashion, etc.  
    product_type    VARCHAR(20) DEFAULT 'Regular', -- Regular or Trial  
    is_available    BOOLEAN DEFAULT true,  
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- NUMERIC(10,2): 10 total digits, 2 after decimal  
-- DEFAULT 'Regular': if you skip product_type, it auto-fills 'Regular'  
-- DEFAULT CURRENT_TIMESTAMP: auto-fills with right now's date+time
```

CREATE TABLE — The Main Event (5/5)

[Example 3 — An Orders Table (the core of any e-commerce DB)]

```
CREATE TABLE transactions.orders (  
  order_id      SERIAL,  
  customer_id   INT,           -- FK (we add this constraint on Constraints & DML)  
  store_id     INT,  
  order_date    DATE DEFAULT CURRENT_DATE,  
  order_status  VARCHAR(30) DEFAULT 'Processing',  
  gross_total   NUMERIC(12, 2), -- before discounts  
  discount     NUMERIC(10, 2) DEFAULT 0,  
  net_total    NUMERIC(12, 2),  -- after discounts  
  payment_mode  VARCHAR(30),    -- UPI, Credit Card, etc.  
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Notice: customer_id and store_id are INT, not SERIAL.  
-- They reference OTHER tables. We link them with FOREIGN KEY on Constraints & DML.  
-- For now, they are just plain INT columns.
```

Naming Conventions — The Unwritten Rules

[Professional Naming Standards]

RULE	Good ✓	Bad ✗
Use snake_case	first_name	firstName, FirstName
Use lowercase only	order_date	Order_Date, ORDER_DATE
Table = plural noun	customers, products	customer, product_tbl
Column = singular	city, order_status	cities, statuses
ID column = table_id	customer_id, product_id	id, cust, pid
Boolean = is_ / has_ prefix	is_active, has_projector	active, projector
Timestamp = _at suffix	created_at, updated_at	creation, update_time
Date = _date or _on suffix	order_date, enrolled_on	date, when
No reserved words	order_status	order, status, type

```
-- Why snake_case?  
-- PostgreSQL internally lowercases everything.  
-- 'OrderDate' becomes 'orderdate' unless you quote it.  
-- 'order_date' stays 'order_date'. No surprises.
```

```
-- RetailMart V3 follows these conventions exactly.  
-- Look at any table: customer_id, first_name, order_date, is_default.
```

CREATE TABLE IF NOT EXISTS — The Safety Net

[Why You Need This]

-- Without IF NOT EXISTS:

```
CREATE TABLE shop.products (product_id SERIAL, name VARCHAR(100));
```

-- Run it once → SUCCESS

-- Run it again → ERROR: relation 'shop.products' already exists

-- With IF NOT EXISTS:

```
CREATE TABLE IF NOT EXISTS shop.products (  
    product_id SERIAL,  
    name VARCHAR(100)  
);
```

-- Run it once → SUCCESS

-- Run it again → NOTICE: relation already exists, skipping

-- No error. Safe to run 100 times.

-- ALWAYS use this in scripts that might be run repeatedly.

-- Production setup scripts, migration scripts, CI/CD pipelines

-- all rely on IF NOT EXISTS for idempotency.

DROP — The Demolition Crew (1/2)

DROP permanently destroys a database object. There is no undo. No recycle bin. No Ctrl+Z. Gone forever.

[The DROP Commands]

-- Drop a table

```
DROP TABLE shop.products;
```

-- Drop a schema (fails if schema has tables)

```
DROP SCHEMA shop;
```

-- ERROR: cannot drop schema shop because other objects depend on it

DROP — The Demolition Crew (2/2)

```
-- Drop schema AND everything inside it
DROP SCHEMA shop CASCADE;
-- CASCADE = 'destroy the folder AND all tables inside'
-- This is DANGEROUS. It deletes ALL tables in the schema.

-- Drop a database (must be connected to a DIFFERENT database)
DROP DATABASE accio_test;

-- Safe versions (no error if it does not exist)
DROP TABLE IF EXISTS shop.products;
DROP SCHEMA IF EXISTS shop CASCADE;
DROP DATABASE IF EXISTS accio_test;
```

DDL vs DML — The Line That Confuses Everyone

Question Students Ask	Answer
'DROP removes data, right?'	DROP removes the TABLE ITSELF + data. DELETE removes ROWS but the table stays.
'TRUNCATE is DDL or DML?'	TRUNCATE is DDL. It removes ALL rows but keeps the table structure intact. Think: flip the table upside down, everything falls out, but the table is still there.
DROP TABLE	→ Table is GONE. Columns gone. Data gone. Everything gone.
TRUNCATE	→ Table EXISTS but is EMPTY. Structure intact. 0 rows.
DELETE FROM	→ Table exists. Some rows removed. Others remain.
DROP	= Demolish the room
TRUNCATE	= Remove ALL furniture from the room (room stays)
DELETE	= Remove SPECIFIC furniture (room + other furniture stays)

PRACTICE BLOCK 1 — DDL: Building Structures

All practice in YOUR database (accio_26). Do NOT touch RetailMart.

SCENARIO: You are the new Database Administrator at Prestige Engineering College. The dean wants to digitize all paper-based records. Start by building the database structure from scratch.

YOUR TASK: Write and execute the following DDL commands in `accio_26`:

a) : Create a schema called college

b) : Create a table college.students with these columns: student_id (auto-increment), first_name (text up to 50 chars), last_name (text up to 50 chars), email (text up to 150 chars), date_of_birth (calendar date), is_active (true/false, defaults to true)

c) : Create a table college.courses with: course_id (auto-increment), course_name (text up to 100 chars), credits (whole number), department (text up to 50 chars)

d) : Verify both tables exist: `SELECT table_schema, table_name FROM information_schema.tables WHERE table_schema = 'college';`

e) : Check the column details: `\d college.students` (in psql) or use pgAdmin's Columns tab

Hint: Use `CREATE TABLE IF NOT EXISTS` for safety. If you make a mistake, `DROP TABLE IF EXISTS` and recreate. That is the beauty of a practice database.

Answer



ANSWER

```
CREATE SCHEMA IF NOT EXISTS college;
```

```
CREATE TABLE college.students (  
    student_id    SERIAL PRIMARY KEY,    -- auto-increment  
    first_name    VARCHAR(50),  
    last_name     VARCHAR(50),  
    email         VARCHAR(150),  
    date_of_birth DATE,  
    is_active     BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE college.courses (  
    course_id     SERIAL PRIMARY KEY,  
    course_name   VARCHAR(100),  
    credits       INT,  
    department    VARCHAR(50)  
);
```

SCENARIO: The college system is growing. The dean now wants to track teachers, classrooms, fee payments, and library books. Design tables that a real college would need.

YOUR TASK:

a) : Create college.teachers with at least 7 columns. Think carefully about each type: teacher_id, first_name, last_name, email, phone, subject_specialty, monthly_salary, joining_date, is_hod (is head of department).

b) : Create college.classrooms with: room_id, building_name, room_number, capacity (how many students fit), has_projector (yes/no), has_ac (yes/no).

c) : Create college.fee_payments with: payment_id, student_id, fee_type (tuition/hostel/library), amount, payment_date, payment_mode, academic_year, semester. What data type for amount? What for academic_year?

d) : Create college.library_books with: book_id, title, author, isbn, published_year, total_copies, available_copies, added_on. Should ISBN be INT or VARCHAR? Why?

e) : List all tables in college schema. How many do you have now?

Hint: For salary use NUMERIC(10,2). For phone use VARCHAR(15) — phone numbers have +91, dashes, and leading zeros. ISBN has dashes and leading zeros too — it is NOT a number you do math on.

Answer (1/2)

✓ ANSWER

```
CREATE TABLE college.teachers (  
    teacher_id      SERIAL PRIMARY KEY,  
    first_name      VARCHAR(50),  
    last_name       VARCHAR(50),  
    email           VARCHAR(150),  
    phone           VARCHAR(15),      -- text: +91, dashes, leading 0s  
    subject_specialty VARCHAR(80),  
    monthly_salary  NUMERIC(10,2),   -- exact money  
    joining_date    DATE,  
    is_hod          BOOLEAN DEFAULT FALSE  
);
```

```
CREATE TABLE college.classrooms (  
    room_id         SERIAL PRIMARY KEY,  
    building_name   VARCHAR(80),  
    room_number     VARCHAR(20),  
    capacity        INT,  
    has_projector   BOOLEAN,  
    has_ac          BOOLEAN  
);
```

```
CREATE TABLE college.fee_payments (  
    --
```

Answer (2/2)

✓ ANSWER

```
payment_id    SERIAL PRIMARY KEY,
student_id    INT,
fee_type      VARCHAR(20),
amount        NUMERIC(10,2),
payment_date  DATE,
payment_mode  VARCHAR(20),
academic_year VARCHAR(9),          -- e.g. '2025-2026'
semester      INT
);

CREATE TABLE college.library_books (
  book_id      SERIAL PRIMARY KEY,
  title        VARCHAR(200),
  author       VARCHAR(120),
  isbn         VARCHAR(20),        -- VARCHAR: dashes + leading 0s
  published_year INT,
  total_copies INT,
  available_copies INT,
  added_on     DATE
);
```

SCENARIO: The Operations Manager at RetailMart says: 'Our supply chain team needs a new tracking system. Build me the schema in the practice database.' You need to design tables without any guidance — just the business requirement.

YOUR TASK:

- a)** : Create a schema called supply in accio_26.
- b)** : Design supply.warehouses — what columns would a warehouse need? Think: ID, name, city, capacity, manager name, is_active, established_date. Write the CREATE TABLE with correct types.
- c)** : Design supply.suppliers — a company that supplies products. What columns? Company name, contact person, email, phone, city, state, registration_date.
- d)** : Design supply.shipments — a shipment from a supplier to a warehouse. What columns? Shipment ID, supplier ID, warehouse ID, product count, shipped date, expected arrival date, actual arrival date, status (In Transit/Delivered/Delayed).
- e)** : After creating all 3 tables, run `\d supply.shipments`. Compare your design to RetailMart's supply_chain.shipments (run `\d supply_chain.shipments` on RetailMart). What did you miss? What did you add that they did not?

Hint: There are no wrong answers in design — only trade-offs. The point is to THINK about every column and type choice, then compare to a production system.

Answer (1/2)

✓ ANSWER

```
CREATE SCHEMA IF NOT EXISTS supply;
```

```
CREATE TABLE supply.warehouses (  
  warehouse_id    SERIAL PRIMARY KEY,  
  name            VARCHAR(100),  
  city            VARCHAR(50),  
  capacity        INT,  
  manager_name    VARCHAR(100),  
  is_active       BOOLEAN DEFAULT TRUE,  
  established_date DATE  
);
```

```
CREATE TABLE supply.suppliers (  
  supplier_id     SERIAL PRIMARY KEY,  
  company_name    VARCHAR(120),  
  contact_person  VARCHAR(100),  
  email           VARCHAR(150),  
  phone           VARCHAR(15),  
  city            VARCHAR(50),  
  state           VARCHAR(50),  
  registration_date DATE  
);
```

Answer (2/2)

✓ ANSWER

```
CREATE TABLE supply.shipments (  
  shipment_id      SERIAL PRIMARY KEY,  
  supplier_id      INT,           -- FK -> suppliers  
  warehouse_id     INT,           -- FK -> warehouses  
  product_count    INT,  
  shipped_date      DATE,  
  expected_arrival_date DATE,  
  actual_arrival_date DATE,  
  status           VARCHAR(20)    -- In Transit/Delivered/Delayed  
);
```

SCENARIO: The CTO says: 'We are building RetailMart V4. I want you to recreate the sales schema from scratch — just by looking at the current V3 design. Prove you understand DDL.'

YOUR TASK:

- a)** : Create a schema called `mini_mart` in `accio_26`.
- b)** : Look at RetailMart's `sales.orders` structure (`\d sales.orders` in `psql`). Recreate it in your schema as `mini_mart.orders` with the same column names and appropriate data types.
- c)** : Do the same for `sales.order_items`. Look at the real table, recreate it.
- d)** : Do the same for `customers.customers`.
- e)** : Now: write a single SQL query that lists ALL columns and data types from your `mini_mart` schema:
- f)** : Run the same query for RetailMart's 'sales' schema. Compare side by side. Where do your types differ?

Hint: `information_schema.columns` is a SYSTEM table that stores metadata about every column in every table. This is how DBAs audit database designs.

Introspection Query

✓ ANSWER

```
SELECT table_name, column_name, data_type, character_maximum_length  
FROM information_schema.columns  
WHERE table_schema = 'mini_mart'  
ORDER BY table_name, ordinal_position;
```

Answer (1/2)

✓ ANSWER

```
CREATE SCHEMA IF NOT EXISTS mini_mart;
```

```
CREATE TABLE mini_mart.orders (  
  order_id      SERIAL PRIMARY KEY,  
  cust_id       INT,  
  store_id      INT,  
  order_date    DATE,  
  order_status  VARCHAR(20),  
  gross_total   NUMERIC(12,2),  
  discount_amount NUMERIC(12,2),  
  net_total     NUMERIC(12,2),  
  payment_mode_id INT  
);
```

```
CREATE TABLE mini_mart.order_items (  
  order_item_id SERIAL PRIMARY KEY,  
  order_id      INT,  
  prod_id       INT,  
  quantity      INT,  
  unit_price    NUMERIC(12,2),  
  gross_amount  NUMERIC(12,2),  
  discount_amount NUMERIC(12,2),
```

Answer (2/2)

✓ ANSWER

```
    net_amount      NUMERIC(12,2)
);

CREATE TABLE mini_mart.customers (
    customer_id      SERIAL PRIMARY KEY,
    first_name       VARCHAR(50),
    last_name        VARCHAR(50),
    email            VARCHAR(100),
    phone            VARCHAR(20),
    registration_date DATE,
    tier              VARCHAR(20),
    tier_updated_at   TIMESTAMP
);
```

The ₹34 Lakh Bug — Why Data Types Are Not Optional (1/2)

True story pattern (names changed). A fintech startup in Bengaluru stored transaction amounts as VARCHAR (text) in their payments table. Their reasoning: 'We will convert to numbers when we need to calculate.'

For 8 months, everything worked. Then:

[How VARCHAR Destroyed Their Finances]

-- Their payment INSERT looked fine:

```
INSERT INTO payments (user_id, amount) VALUES (1001, '15000');
```

```
INSERT INTO payments (user_id, amount) VALUES (1002, '7500.50');
```

-- But one day, a frontend bug sent this:

```
INSERT INTO payments (user_id, amount) VALUES (1003, '₹15,000');
```

```
INSERT INTO payments (user_id, amount) VALUES (1004, 'fifteen thousand');
```

```
INSERT INTO payments (user_id, amount) VALUES (1005, '15000.00.00');
```

The ₹34 Lakh Bug — Why Data Types Are Not Optional (2/2)

```
-- VARCHAR accepted ALL of these. No error. No warning.  
-- The data sat in the database for 3 months unnoticed.  
  
-- Quarter-end report:  
SELECT SUM(amount::NUMERIC) FROM payments;  
-- ERROR: invalid input syntax for type numeric: '₹15,000'  
  
-- 34 lakh rupees worth of transactions had corrupted amount values.  
-- 3 engineers spent 2 weeks cleaning the data manually.  
-- The fix? One line:  
  
ALTER TABLE payments ALTER COLUMN amount TYPE NUMERIC(12,2);  
  
-- Now '₹15,000' is REJECTED at INSERT time.  
-- The bug would have been caught immediately, not 8 months later.
```

Data types are not bureaucratic rules. They are your **FIRST** line of defense. The database rejects bad data at the gate — so you never have to clean it later. Choose the right type, and PostgreSQL protects you 24/7 for free.

Integer Types — Whole Numbers Without Decimals

Type	Storage	Range	Real-World Use
SMALLINT	2 bytes	-32,768 to 32,767	Age, rating (1-5), floor number
INT/INTEGER	4 bytes	-2.1 billion to 2.1 billion	Most IDs, quantities, counts
BIGINT	8 bytes	±9.2 quintillion	Instagram likes, YouTube views

How to choose:

age	→ SMALLINT	(humans live to ~120, 32K is plenty)
quantity	→ INT	(nobody orders 2 billion units)
employee_count	→ INT	(even TCS has only ~600K employees)
youtube_views	→ BIGINT	(Despacito has 8+ billion views)

Rule: Use the SMALLEST type that fits your data.

SMALLINT uses 2 bytes. INT uses 4. BIGINT uses 8.

Over 50,000 rows, wrong choice wastes megabytes.

Over 50 million rows, it wastes gigabytes.

Auto-Increment Types — IDs That Count Themselves (1/2)

Type	Underlying	What It Does
SERIAL	INT	Auto-generates: 1, 2, 3, 4, 5...
BIGSERIAL	BIGINT	Same, for tables expecting billions of rows

- How SERIAL works under the hood:
- PostgreSQL creates a hidden SEQUENCE object
- Each INSERT calls nextval() automatically
- You NEVER insert the SERIAL column manually

```
CREATE TABLE public.students (  
    student_id SERIAL,          -- auto-incremented  
    name          VARCHAR(100)  
);  
  
INSERT INTO students (name) VALUES ('Rahul');    -- student_id = 1  
INSERT INTO students (name) VALUES ('Priya');    -- student_id = 2  
INSERT INTO students (name) VALUES ('Amit');     -- student_id = 3  
  
-- DELETE Priya (student_id = 2)
```

Auto-Increment Types — IDs That Count Themselves (2/2)

```
DELETE FROM students WHERE student_id = 2;
```

```
INSERT INTO students (name) VALUES ('Sneha');    -- student_id = 4 (NOT 2!)  
-- SERIAL never reuses deleted IDs. 2 is gone forever.  
-- This is intentional – prevents confusion in audit logs.
```

Decimal Types — Money, Prices, and Precision

Type	Precision	Use When
NUMERIC(p,s)	Exact	Money, prices, tax – ALWAYS
REAL	~6 digits (approx)	Scientific measurements only
DOUBLE PREC.	~15 digits (approx)	Scientific calculations only

NUMERIC(p, s) explained:

p = precision = TOTAL digits (before + after decimal)

s = scale = digits AFTER the decimal point

NUMERIC(10, 2) → max 99,99,99,999.99 (10 digits, 2 after dot)

NUMERIC(5, 2) → max 999.99 (5 digits, 2 after dot)

NUMERIC(8, 4) → max 9999.9999 (8 digits, 4 after dot)

RetailMart V3 usage:

price, cost_price	→ NUMERIC(10, 2)	– up to ₹99,99,99,999.99
discount_percent	→ NUMERIC(5, 2)	– up to 999.99%
net_total	→ NUMERIC(12, 2)	– order totals can be large
latitude, longitude	→ NUMERIC(9, 6)	– GPS precision to ~11cm

CRITICAL: Never Use FLOAT/REAL for Money

[The Rounding Horror]

```
-- FLOAT/REAL store numbers in binary floating-point format
-- This causes tiny rounding errors:
```

```
SELECT 0.1::REAL + 0.2::REAL;
-- Returns: 0.30000001192092896 ← WRONG!
```

```
SELECT 0.1::NUMERIC + 0.2::NUMERIC;
-- Returns: 0.3 ← CORRECT
```

```
-- 'But 0.00000001 is tiny, who cares?'
--
-- Calculate: 150,000 orders × ₹0.00000001 error per order
-- = ₹0.0015 per batch. Harmless?
--
-- Now: 150,000 orders/day × 365 days × 10 years × compound
-- Banks lost MILLIONS to accumulated floating-point drift.
-- Knight Capital lost $440 MILLION in 45 minutes partly due to
-- a software bug involving numeric precision.
```

```
-- The rule is absolute: NUMERIC for money. No exceptions. Ever.
```

The Three Text Types

Type	Max Length	Storage Behavior	Best For
CHAR(n)	Fixed n	ALWAYS uses n bytes (pads)	Almost never. Legacy only.
VARCHAR(n)	Up to n	Uses only what it needs	Names, emails, cities, codes
TEXT	Unlimited	Uses only what it needs	Reviews, descriptions, comments

CHAR(10) storing 'hello':

- Stored as 'hello ' (5 chars + 5 spaces = always 10 bytes)
- Wastes 50% storage on this row

VARCHAR(100) storing 'hello':

- Stored as 'hello' (just 5 bytes + 1 byte length header)
- No waste. Efficient.

TEXT storing a 5000-word product review:

- Stored as-is. No upper limit.
- Perfect for long-form content.

Bottom line: VARCHAR for most things. TEXT for long content. Forget CHAR exists.

Practical VARCHAR Sizes — How Long Is Each?

Column	Type	Reasoning
first_name	VARCHAR(50)	Longest Indian names ~30 chars
last_name	VARCHAR(50)	Same reasoning
email	VARCHAR(150)	Emails can be surprisingly long
phone	VARCHAR(15)	+91-98765-43210 = 15 chars
city	VARCHAR(100)	Thiruvananthapuram = 18 chars
state	VARCHAR(50)	Andaman and Nicobar Islands = 28
pincode	VARCHAR(10)	6 digits in India, 10 max globally
product_name	VARCHAR(200)	Product titles on Amazon get LONG
order_status	VARCHAR(30)	'Out for Delivery' = 16 chars
payment_mode	VARCHAR(30)	'Credit Card' = 11 chars
review_text	TEXT	No limit — customers write essays
address_line	VARCHAR(250)	Indian addresses are verbose

Pro tip: Better to set VARCHAR slightly too large than too small.
VARCHAR(100) and VARCHAR(50) use the same storage for a 30-char string.
The limit only rejects data that EXCEEDS it.

The Phone Number & Pincode Trap (1/2)

[Why These Are NEVER INT]

```
-- Phone: +91-098765-43210
--   Has +, dashes, spaces → not a valid integer
--   Leading zero: 098765... → INT strips it → 98765...

-- Pincode: 022987
--   Leading zero → INT stores as 22987 → WRONG pincode!
--   Mumbai pincodes start with 4000xx
--   Delhi pincodes start with 1100xx
```

The Phone Number & Pincode Trap (2/2)

```
-- Aadhaar: 0001-2345-6789
--   Leading zeros → INT → 123456789 → missing 3 digits!

-- PAN: ABCDE1234F
--   Has letters → definitely not INT

-- The Golden Rule:
-- If you NEVER do math on it (SUM, AVG, multiply),
-- it is NOT a number. Store it as VARCHAR.

-- You never calculate: SUM(phone_number) or AVG(pincode).
-- You DO calculate: SUM(salary) and AVG(price).
-- That is the test.
```

Date/Time Types — Complete Guide (1/2)

Type	What It Stores	Example	Use When
DATE	Calendar date only	'2025-05-16'	Birthdays, order dates
TIME	Clock time only	'14:30:00'	Schedules, shifts
TIMESTAMP	Date + Time (no timezone)	'2025-05-16 14:30:00'	created_at, events
TIMESTAMPZ	Date + Time + Timezone	'2025-05-16 14:30:00+05:30'	Global apps
INTERVAL	Duration / time span	'3 days 4 hours 20 mins'	SLA tracking

The format PostgreSQL expects: YYYY-MM-DD (ISO 8601)

- ✓ '2025-05-16' (correct)
- ✗ '16/05/2025' (DD/MM/YYYY – Indian format, rejected)
- ✗ '05/16/2025' (MM/DD/YYYY – American format, ambiguous)

When to use DATE vs TIMESTAMP:

- date_of_birth → DATE (time of birth rarely matters)
- order_date → DATE (the DAY matters, not the hour)
- joining_date → DATE (which day they joined)
- created_at → TIMESTAMP (exact moment a record was created)
- login_time → TIMESTAMP (exact moment user logged in)
- check_in → TIMESTAMP (exact clock-in time for attendance)

Date/Time Types — Complete Guide (2/2)

`page_view_at` → `TIMESTAMP` (exact moment user viewed a page)

INTERVAL — The Underrated Type

```
-- INTERVAL stores DURATIONS, not moments in time.

-- How long was the support call?
call_duration    INTERVAL    -- '00:07:23' (7 mins 23 secs)

-- When does the warranty expire?
SELECT purchase_date + INTERVAL '2 years' AS warranty_expiry
FROM orders;

-- What is the average delivery time?
SELECT AVG(delivered_date - shipped_date) AS avg_delivery_time
FROM shipments;

-- Result: '3 days 14:22:00' – 3.5 days average

-- RetailMart V3 does not use INTERVAL directly,
-- but you will CALCULATE intervals in queries starting Scalar Functions.
-- Understanding the type now saves confusion later.
```

BOOLEAN — True, False, or NULL

Type	Values	Storage	Use When
BOOLEAN	true / false / NULL	1 byte	Yes/No flags

-- In RetailMart V3:

-- customers.addresses.is_default → BOOLEAN (is this the primary address?)

-- products.promotions.active → BOOLEAN (is this promo running?)

-- Common patterns:

is_active BOOLEAN DEFAULT true

email_verified BOOLEAN DEFAULT false

has_projector BOOLEAN

is_veg BOOLEAN

-- Why NOT use VARCHAR for yes/no?

-- VARCHAR allows: 'yes', 'Yes', 'YES', 'Y', 'true', 'TRUE', 'si', 'maybe'

-- BOOLEAN allows: true, false, NULL. That is it. No ambiguity.

-- Filtering: WHERE is_active = true vs WHERE status = 'active'

-- BOOLEAN is faster, cleaner, and less error-prone.

Special Types — Preview (Deep Dive Later in the Course)

Type	What It Stores	Course Day	Why It Exists
UUID	Universally unique ID (128-bit)	Mentioned	Unpredictable IDs for APIs
JSONB	Binary JSON data	JSON	Flexible nested data
ARRAY	List of values in one cell	JSON	Tags, categories, multi-value

UUID example: 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11'

- Unlike SERIAL (1, 2, 3...), UUIDs are random.
- Users cannot guess IDs. Crucial for API security.
- SERIAL: /api/user/1 → try /api/user/2 → works! (insecure)
- UUID: /api/user/a0eebc99... → unguessable (secure)

JSONB example: '{"tags": ["electronics", "sale"], "specs": {"ram": "8GB"}}'

- When some products have 'ram' and others have 'fabric_type',
- rigid columns do not work. JSONB stores flexible attributes.
- We cover this deeply on JSON.

For now: 95% of your columns will be SERIAL, INT, NUMERIC, VARCHAR, TEXT, DATE, TIMESTAMP, or BOOLEAN. Master these 8, and you handle 95% of cases.

The 10-Second Decision Tree (1/2)

Ask yourself these questions in order:

1. Is it an auto-incrementing ID?
YES → SERIAL (or BIGSERIAL for huge tables)
2. Is it a whole number I will do math on? (quantity, count, age)
YES → SMALLINT / INT / BIGINT (pick smallest that fits)
3. Is it money or a precise decimal?
YES → NUMERIC(p, s) – NEVER FLOAT/REAL
4. Is it a short text? (name, email, city, code)
YES → VARCHAR(n)
5. Is it a long text? (review, description, bio)
YES → TEXT
6. Is it a calendar date? (birthday, order date)
YES → DATE

The 10-Second Decision Tree (2/2)

7. Is it an exact moment? (login, created_at, event time)

YES → TIMESTAMP

8. Is it yes/no?

YES → BOOLEAN

9. I NEVER do math on it, but it looks like a number? (phone, pin, aadhaar)

→ VARCHAR(n) – NOT INT

Still confused? Default to VARCHAR. You can always ALTER later.

But going from VARCHAR→NUMERIC is painful if bad data already exists.

REAL WORLD (1/2)

RETAILMART V3 TYPE AUDIT — Learn From Production

Let us look at how RetailMart V3 chose types for real columns:

[customers.customers – Type Choices Explained]

Column	Type	Why This Type?
customer_id	SERIAL (INT)	Auto-increment, primary key
first_name	VARCHAR(50)	Names rarely exceed 30 chars
last_name	VARCHAR(50)	Same reasoning
email	VARCHAR(150)	Emails can be long (company domains)
phone	VARCHAR(15)	NOT INT – has +91, dashes, leading zeros
registration_date	DATE	Only the day matters, not the time
tier	VARCHAR(20)	'Silver', 'Gold', 'Platinum', 'Diamond'
tier_updated_at	TIMESTAMP	Exact moment tier changed – matters for analytics

REAL WORLD (2/2)

[sales.orders – Type Choices Explained]

Column	Type	Why This Type?
order_id	SERIAL	Auto-increment PK
cust_id	INT	References customers.customer_id (FK)
store_id	INT	References stores.store_id (FK)
order_date	DATE	Which day – not the exact hour
order_status	VARCHAR(30)	'Delivered', 'Processing', 'Cancelled'...
gross_total	NUMERIC(12,2)	Money – MUST be exact. Allows up to ₹99,99,99,99,999.99
discount	NUMERIC(10,2)	Discount amount in rupees
net_total	NUMERIC(12,2)	gross_total - discount
payment_mode_id	INT	References finance.payment_modes (FK)

Notice the patterns: IDs are SERIAL or INT. Money is always NUMERIC. Dates that need time use TIMESTAMP. Status fields are VARCHAR. Every choice is deliberate.

stores.employees — Type Choices for People Data

Column	Type	Why This Type?
employee_id	SERIAL	Auto-increment PK
store_id	INT (nullable)	FK → stores. NULL for HQ/Regional staff
first_name	VARCHAR(50)	Names rarely exceed 30 chars
last_name	VARCHAR(50)	Same reasoning
email	VARCHAR(100)	Unique work email
role	VARCHAR(50)	'Store Manager', 'Cashier', 'Data Analyst'...
dept_id	INT	FK → core.dim_department
joining_date	DATE	Which day – time irrelevant
salary	INT	Monthly salary in INR (whole rupees)

Key insight: store_id is NULLABLE because Regional Managers and HQ staff (Data Engineers, Finance Executives) do not belong to a specific store.

Note: hierarchy here is INFERRED from role (e.g. 'Store Manager' supervises non-manager employees at the same store) rather than a manager_id column.

web_events.page_views — Type Choices for Event Data

Column	Type	Why This Type?
view_id	SERIAL	Auto-increment PK
session_id	VARCHAR(50)	Browser session hash – NOT INT (contains letters)
customer_id	INT (nullable)	FK. NULL for anonymous/logged-out visitors
page_url	VARCHAR(500)	URLs can be very long with query params
view_timestamp	TIMESTAMP	Exact moment – not just the date
device_type	VARCHAR(20)	'Mobile', 'Desktop', 'Tablet'
os	VARCHAR(30)	'Android', 'iOS', 'Windows', 'macOS'

Why TIMESTAMP here but DATE for orders?

Orders: 'Rahul ordered on May 15' → the DAY matters

Page views: 'Rahul viewed the homepage at 14:32:07' → the SECOND matters

Analytics needs: 'How many views per HOUR?' → impossible with DATE

Design Decisions Are Trade-offs, Not Rules (1/2)

Students often ask: 'What is the RIGHT answer for this column type?' The truth: there are often multiple valid choices. What matters is your REASONING.

[Real Trade-offs in RetailMart V3]

Trade-off 1: `order_status` as `VARCHAR(30)` vs a `status_id` `INT` `FK`
`VARCHAR` → Easy to read: 'Delivered', 'Processing', 'Cancelled'
`INT FK` → Faster filtering, no typos, enforced vocabulary
RetailMart chose `VARCHAR` for simplicity. Large systems use `INT FK`.

Design Decisions Are Trade-offs, Not Rules (2/2)

Trade-off 2: `product_type` as `VARCHAR(20)` vs `BOOLEAN`

`BOOLEAN` → Only 2 values possible (true/false)

`VARCHAR` → Can expand: 'Regular', 'Trial', 'Seasonal', 'Discontinued'

RetailMart chose `VARCHAR` for future flexibility. Smart call.

Trade-off 3: `salary` as `NUMERIC(10,2)` vs `INT`

`INT` → No paisa. ₹45,000 works. ₹45,000.75 fails.

`NUMERIC(10,2)` → Exact to paisa. Overkill for most Indian salaries?

RetailMart chose `NUMERIC` because payroll calculations need precision.

Trade-off 4: `phone` as `VARCHAR(15)` vs separate `country_code` + `number`

Single column → Simple but messy ('+91-98765-43210')

Split columns → Clean but more `JOINS` needed

RetailMart chose single column. Phone normalization is Data Quality topic.

The interviewer does not want THE answer. They want YOUR reasoning.

PRACTICE BLOCK 2 — Data Types

Focus on choosing the RIGHT type. Every wrong type is a future bug.

SCENARIO: The HR Director at RetailMart asks you to create a new table for tracking employee performance reviews in the practice database.

YOUR TASK: Create college.performance_reviews (or a new table) with these columns. Choose the CORRECT data type for each:

- a)** : review_id — auto-incrementing ID
- b)** : employee_id — references an employee
- c)** : reviewer_name — who conducted the review
- d)** : review_date — when the review happened
- e)** : rating — a score from 1 to 5 (can be 3.5)
- f)** : comments — detailed written feedback (can be very long)
- g)** : salary_hike_percent — percentage raise recommended (e.g., 12.50%)
- h)** : is_promoted — whether the employee was promoted
- i)** : next_review_date — when the next review is scheduled

Hint: rating 3.5 means it has a decimal → NUMERIC. comments could be 1000 words → TEXT. salary_hike_percent 12.50 → NUMERIC(5,2).

Answer

✓ ANSWER

```
CREATE TABLE college.performance_reviews (  
  review_id          SERIAL PRIMARY KEY,  -- auto-increment  
  employee_id        INT,                 -- references an employee  
  reviewer_name       VARCHAR(100),  
  review_date        DATE,  
  rating             NUMERIC(2,1),        -- 1.0-5.0, allows 3.5  
  comments           TEXT,                -- can be very long  
  salary_hike_percent NUMERIC(5,2),        -- e.g. 12.50  
  is_promoted         BOOLEAN,  
  next_review_date    DATE  
);
```

SCENARIO: A junior developer sent you this CREATE TABLE script. It is FULL of wrong data type choices. Find and fix EVERY mistake.

YOUR TASK:

- a) : For EACH column, explain WHAT is wrong and WHY.
- b) : Write the CORRECTED CREATE TABLE statement.
- c) : Execute the corrected version in `accio_26`.
- d) : Insert 3 sample rows to verify your types work. (Use `INSERT INTO ... VALUES ...`)

Hint: Bug 4: ₹1,299.50 has paisa — INT truncates to ₹1,299. Bug 8: Rating is 1-5 — NUMERIC(10,5) allows 99999.99999 — absurd overkill.

Find the Bugs — 9 Wrong Types



```
CREATE TABLE public.customer_orders (  
    order_id      VARCHAR(10),      -- Bug 1?  
    customer_name CHAR(200),        -- Bug 2?  
    order_date    VARCHAR(20),      -- Bug 3?  
    total_amount  INT,              -- Bug 4?  
    phone         BIGINT,           -- Bug 5?  
    pincode       INT,              -- Bug 6?  
    is_delivered  VARCHAR(5),       -- Bug 7?  
    rating        NUMERIC(10,5),    -- Bug 8?  
    notes         VARCHAR(50)       -- Bug 9?  
);
```

Answer



```
CREATE TABLE customer_orders (  
  order_id      SERIAL PRIMARY KEY, -- was VARCHAR(10): IDs are integers  
  customer_name VARCHAR(100),        -- was CHAR(200): CHAR pads; names vary  
  order_date    DATE,                -- was VARCHAR(20): use a real date  
  total_amount  NUMERIC(12,2),        -- was INT: money needs paisa  
  phone         VARCHAR(15),          -- was BIGINT: keeps +91, 0s, dashes  
  pincode       VARCHAR(6),           -- was INT: keeps leading zeros  
  is_delivered  BOOLEAN,              -- was VARCHAR(5): true/false  
  rating        NUMERIC(2,1),          -- was NUMERIC(10,5): rating is 1-5  
  notes         TEXT                  -- was VARCHAR(50): notes can be long  
);
```

SCENARIO: You are designing a database for Zomato from scratch. Millions of orders per day. Every wrong type choice will haunt you at scale.

YOUR TASK: Write CREATE TABLE statements for these 4 tables. Justify each type:

a) : restaurants — restaurant_id, name, city, latitude, longitude, avg_rating, total_reviews, is_pure_veg, cuisine_type, opening_time, closing_time, created_at. (What type for lat/long — 28.6139°N needs 4 decimal places minimum? What type for avg_rating — 4.3 is possible?)

b) : menu_items — item_id, restaurant_id, item_name, description, price, is_veg, prep_time_minutes, calories. (description can be 500 words. calories is a whole number. price in rupees.)

c) : delivery_orders — order_id, customer_id, restaurant_id, order_time, delivery_time, total_amount, delivery_fee, distance_km, tip_amount, payment_method, order_status. (distance can be 3.7 km. order_time needs exact timestamp.)

d) : reviews — review_id, order_id, customer_id, rating, review_text, photo_url, review_date, is_edited, helpful_votes. (photo_url can be very long. helpful_votes is a count.)

Hint: lat/long → NUMERIC(9,6) gives precision to ~11cm. rating → NUMERIC(2,1) allows 4.3 but blocks 42.3. distance → NUMERIC(5,2) allows 999.99 km.

Answer (1/3)

✓ ANSWER

```
CREATE TABLE restaurants (  
  restaurant_id SERIAL PRIMARY KEY,  
  name          VARCHAR(150),  
  city          VARCHAR(50),  
  latitude      NUMERIC(9,6),      -- ~11cm precision  
  longitude     NUMERIC(9,6),  
  avg_rating    NUMERIC(2,1),      -- 4.3 ok, blocks 42.3  
  total_reviews INT,  
  is_pure_veg   BOOLEAN,  
  cuisine_type  VARCHAR(60),  
  opening_time  TIME,  
  closing_time  TIME,  
  created_at    TIMESTAMPTZ DEFAULT now()  
);
```

```
CREATE TABLE menu_items (  
  item_id      SERIAL PRIMARY KEY,  
  restaurant_id INT,  
  item_name     VARCHAR(120),  
  description   TEXT,          -- up to 500 words  
  price        NUMERIC(10,2),  
  is_veg       BOOLEAN,
```

Answer (2/3)

✓ ANSWER

```
    prep_time_minutes INT,  
    calories          INT  
);
```

```
CREATE TABLE delivery_orders (  
    order_id          SERIAL PRIMARY KEY,  
    customer_id       INT,  
    restaurant_id     INT,  
    order_time        TIMESTAMPTZ,      -- exact timestamp  
    delivery_time     TIMESTAMPTZ,  
    total_amount      NUMERIC(12,2),  
    delivery_fee      NUMERIC(8,2),  
    distance_km       NUMERIC(5,2),     -- 3.7 km  
    tip_amount        NUMERIC(8,2),  
    payment_method    VARCHAR(20),  
    order_status      VARCHAR(20)  
);
```

```
CREATE TABLE reviews (  
    review_id         SERIAL PRIMARY KEY,  
    order_id          INT,  
    customer_id       INT,
```

Answer (3/3)

✓ ANSWER

```
rating      NUMERIC(2,1),
review_text TEXT,
photo_url   TEXT,          -- can be very long
review_date DATE,
is_edited   BOOLEAN,
helpful_votes INT
);
```

SCENARIO: The CTO of RetailMart gives you a challenge: 'Audit 5 of our tables. For each table, check every column type. Tell me if any type choice is wrong, suboptimal, or could be improved. I want a written report.'

YOUR TASK:

a) : Run `\d customers.customers` in psql on RetailMart. List every column + type. For at least 5 columns, explain WHY that type was chosen. Is any choice debatable?

b) : Run `\d sales.orders`. Why is `order_date` `DATE` and not `TIMESTAMP`? Would `TIMESTAMP` have been better? Argue both sides.

c) : Run `\d products.products`. What type is `product_type`? Why `VARCHAR(20)` and not `BOOLEAN` (since it is only 'Regular' or 'Trial')? When is `VARCHAR` better than `BOOLEAN` for a 2-value column?

d) : Run `\d stores.employees`. What type is `salary`? What about `manager_id` — why is it nullable (allows `NULL`)? Which employees have `NULL manager_id` and why?

e) : Find ONE column across any RetailMart table where you would make a DIFFERENT type choice. Write the `ALTER TABLE` statement to implement your change. Defend your decision.

Hint: `product_type` as `VARCHAR` allows future expansion: 'Regular', 'Trial', 'Seasonal', 'Limited Edition'. `BOOLEAN` only allows 2 values forever. That is a real-world trade-off.

Answer



- a) customers.customers: customer_id INT (surrogate PK), email VARCHAR(100) (bounded), phone VARCHAR(20) (keeps +91/0s), registration_date DATE, tier VARCHAR(20) (extensible). Debatable: tier could be an FK to a table.
- b) sales.orders.order_date is DATE because reporting is by calendar day; TIMESTAMP would help only if intraday timing mattered (it does not here).
- c) products.product_type is VARCHAR(20), not BOOLEAN, so it can grow beyond 'Regular'/'Trial' (e.g. 'Seasonal') with no schema change.
- d) stores.employees.salary is INT (whole rupees); manager_id is nullable because top-of-hierarchy staff (Regional Managers/HQ) have no manager.
- e) Example change (illustrative):
ALTER TABLE customers.customers ALTER COLUMN phone TYPE VARCHAR(20);

How RetailMart's Tables Evolved Over Time (1/2)

RetailMart V1 was built 2 years ago. Version V3 (which you are using) has columns that V1 never had. Here is how the products table evolved:

[The Evolution of products.products]

Version 1 (2 years ago):

```
CREATE TABLE products.products (  
    product_id    SERIAL,  
    product_name  VARCHAR(200),  
    brand_id      INT,  
    supplier_id   INT,  
    price         NUMERIC(10,2),  
    cost_price    NUMERIC(10,2)  
);
```

How RetailMart's Tables Evolved Over Time (2/2)

Month 3 – Marketing wanted to distinguish trial products:

```
ALTER TABLE products.products  
ADD COLUMN product_type VARCHAR(20) DEFAULT 'Regular';
```

Month 7 – Logistics needed weight for shipping costs:

```
ALTER TABLE products.products  
ADD COLUMN weight_kg NUMERIC(6,2);
```

Month 12 – Old 'brand' TEXT column replaced with FK to dim_brand:

```
ALTER TABLE products.products DROP COLUMN brand;  
ALTER TABLE products.products ADD COLUMN brand_id INT;
```

V3 (now) – 8 columns. Started as 6.

No table was dropped and recreated. Just ALTER. Data preserved.

This is normal. At Flipkart, engineers run ALTER TABLE multiple times per week. Requirements change. Business grows.

The database evolves. ALTER TABLE is your evolution tool.

ALTER TABLE — The Complete Toolkit (1/2)

-- == ADD A COLUMN ==

```
ALTER TABLE shop.products ADD COLUMN weight_kg NUMERIC(6,2);
```

```
ALTER TABLE shop.products ADD COLUMN color VARCHAR(30) DEFAULT 'Unknown';
```

-- Existing rows get NULL (or the DEFAULT value) for the new column.

-- == DROP A COLUMN ==

```
ALTER TABLE shop.products DROP COLUMN color;
```

-- The column AND all data in it are permanently gone. No undo.

-- == RENAME A COLUMN ==

```
ALTER TABLE shop.products RENAME COLUMN weight_kg TO weight_grams;
```

-- All existing data stays. Just the label changes.

-- Any query using the old name will break. Update your queries too.

-- == CHANGE DATA TYPE ==

```
ALTER TABLE shop.products ALTER COLUMN weight_grams TYPE NUMERIC(8,2);
```

-- If types are incompatible and data exists, use USING:

```
ALTER TABLE shop.products ALTER COLUMN price TYPE INT  
    USING price::INT;
```

ALTER TABLE — The Complete Toolkit (2/2)

```
-- This explicitly casts existing values. May lose decimal precision!
```

```
-- == SET / DROP DEFAULT ==
```

```
ALTER TABLE shop.products ALTER COLUMN color SET DEFAULT 'Unknown';
```

```
ALTER TABLE shop.products ALTER COLUMN color DROP DEFAULT;
```

```
-- SET DEFAULT only affects FUTURE inserts. Existing NULLs stay NULL.
```

```
-- == RENAME THE TABLE ==
```

```
ALTER TABLE shop.products RENAME TO items;
```

```
-- All data preserved. The table just has a new name.
```

```
-- == MOVE TABLE TO DIFFERENT SCHEMA ==
```

```
ALTER TABLE shop.items SET SCHEMA transactions;
```

```
-- The table physically moves from shop to transactions schema.
```

ALTER TABLE Gotchas (1/2)

[Things That Will Trip You Up]

- 1. ADD COLUMN position
- PostgreSQL ALWAYS adds new columns at the END.
- You CANNOT say 'ADD COLUMN middle_name AFTER first_name'.
- (MySQL allows this. PostgreSQL does not.)
- If column order matters, you must recreate the table.

ALTER TABLE Gotchas (2/2)

```
-- 2. ALTER TYPE with incompatible data
-- If column has 'hello' and you ALTER to INT:
ALTER TABLE t ALTER COLUMN c TYPE INT;
-- ERROR: cannot cast type character varying to integer
-- Fix: clean the data first, THEN alter the type.

-- 3. DROP COLUMN with dependencies
-- If a VIEW or INDEX depends on the column:
ALTER TABLE t DROP COLUMN c;
-- ERROR: cannot drop column c because other objects depend on it
-- Fix: DROP the dependent objects first, or use CASCADE:
ALTER TABLE t DROP COLUMN c CASCADE;

-- 4. SET DEFAULT does NOT backfill
ALTER TABLE t ALTER COLUMN c SET DEFAULT 'N/A';
-- Existing NULL values stay NULL. Only FUTURE inserts get 'N/A'.
-- To update existing NULLs: UPDATE t SET c = 'N/A' WHERE c IS NULL;
```

What is Casting? — Preview for Future Days (1/2)

Sometimes you need to convert a value from one type to another on the fly — without changing the table structure. This is called CASTING. You will use it heavily from Filtering & Sorting onwards.

What is Casting? — Preview for Future Days (2/2)

[Two Ways to Cast in PostgreSQL]

-- Method 1: PostgreSQL-style (double colon)

```
SELECT '42'::INT;           -- text → integer → 42
SELECT 42::TEXT;           -- integer → text → '42'
SELECT '2025-05-16'::DATE;  -- text → date
SELECT 99.7::INT;           -- decimal → integer → 99 (truncated!)
SELECT true::INT;          -- boolean → integer → 1
```

-- Method 2: SQL standard (CAST function)

```
SELECT CAST('42' AS INT);   -- same as '42'::INT
SELECT CAST(99.7 AS INT);   -- same as 99.7::INT → 99
```

-- Real-world usage (you will do this from Filtering & Sorting+):

-- When a column is VARCHAR but you need to sort it as a number:

```
SELECT * FROM products ORDER BY price::NUMERIC;
```

-- When comparing dates stored as text (bad design, but you inherit it):

```
SELECT * FROM old_system WHERE order_date::DATE > '2025-01-01';
```

THE TYPE MISTAKES HALL OF SHAME

Every mistake here has been made by real developers at real companies. Learn from their pain.

Mistake 1: Storing Dates as VARCHAR

```
-- What they did:
CREATE TABLE events (event_date VARCHAR(20));
INSERT INTO events VALUES ('15/05/2025');
INSERT INTO events VALUES ('2025-05-15');
INSERT INTO events VALUES ('May 15, 2025');
INSERT INTO events VALUES ('yesterday');

-- ALL accepted. Four different formats. One is not even a date.
-- Now try: SELECT * FROM events WHERE event_date > '2025-01-01';
-- This does ALPHABETICAL comparison, not chronological.
-- '15/05/2025' > '2025-01-01' → true (alphabetically 1 < 2)
-- But that is WRONG – it is comparing characters, not dates.

-- The fix: event_date DATE
-- PostgreSQL enforces YYYY-MM-DD format.
-- Comparisons work correctly. Date math works.
-- 'yesterday' is rejected immediately.
```

Mistake 2: Using INT for Currency

```
-- What they did:
CREATE TABLE invoices (amount INT);
INSERT INTO invoices VALUES (1299);  -- meant ₹1,299.50

-- The 50 paisa? Gone. INT truncates decimals silently.
-- Over 150,000 invoices, total error = ₹75,000 of missing paisa.
-- Nobody noticed for 6 months until the auditor found a mismatch.

-- The fix: amount NUMERIC(12, 2)
-- Stores ₹1,299.50 exactly. No truncation. No rounding.
```

Mistake 3: Using TEXT for Everything

```
-- The lazy developer's approach:
CREATE TABLE products (
    product_id TEXT,    -- should be SERIAL
    name TEXT,          -- OK, but no length limit
    price TEXT,         -- disaster for calculations
    quantity TEXT,     -- cannot do quantity > 0 check
    is_active TEXT      -- 'yes', 'true', 'Y', '1', 'active'...
);

-- Problems:
-- 1. SUM(price) → ERROR (text, not numeric)
-- 2. quantity > 0 → compares alphabetically ('9' > '10' → true!)
-- 3. is_active has 6 different representations of 'true'
-- 4. No auto-increment on product_id – manual management
-- 5. No type safety – anything goes in any column

-- TEXT for everything = no defense against bad data.
-- The database becomes a glorified notepad.
```

Mistake 4: Wrong Precision on NUMERIC

```
-- Scenario: GPS coordinates
CREATE TABLE locations (lat NUMERIC(4,2), lng NUMERIC(4,2));
-- NUMERIC(4,2) → max 99.99
-- Latitude 28.6139 → TRUNCATED to 28.61
-- That is a 300-meter error. Your delivery driver goes to the wrong building.

-- Fix: NUMERIC(9,6) → allows 999.999999
-- 6 decimal places = precision to ~11 centimeters. GPS-grade accuracy.

-- Scenario: Large order totals
CREATE TABLE orders (total NUMERIC(6,2));
-- NUMERIC(6,2) → max 9,999.99
-- A corporate order for ₹1,25,000? → ERROR: numeric field overflow

-- Fix: NUMERIC(12,2) → max 99,99,99,99,999.99
-- Think about the MAXIMUM value this column could ever hold.
```

Mistake 5: VARCHAR Too Short

```
-- What they did:  
CREATE TABLE customers (email VARCHAR(30));  
  
-- This broke on:  
-- 'ramachandran.krishnamurthy@prestigious-university.ac.in'  
-- That is 57 characters. ERROR: value too long for type character varying(30)  
  
-- Customer could not register. Support ticket filed. Hot fix deployed.  
-- All because someone assumed emails are short.  
  
-- The fix: email VARCHAR(150)  
-- Generous limit. VARCHAR(150) and VARCHAR(30) use identical storage  
-- for a 25-character email. The limit only rejects excess.  
-- There is NO performance penalty for a larger VARCHAR limit.
```

How to Inspect Any Table's Design (1/2)

-- List all tables in a schema:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'college'
ORDER BY table_name;
```

-- List all columns of a specific table with their types:

```
SELECT column_name, data_type, character_maximum_length,
       numeric_precision, numeric_scale, column_default, is_nullable
FROM information_schema.columns
WHERE table_schema = 'customers' AND table_name = 'customers'
ORDER BY ordinal_position;
```

-- Count tables per schema (like a database inventory):

```
SELECT table_schema, COUNT(*) AS table_count
FROM information_schema.tables
WHERE table_schema NOT IN ('pg_catalog', 'information_schema')
GROUP BY table_schema
ORDER BY table_count DESC;
```

How to Inspect Any Table's Design (2/2)

- This is how DBAs audit database designs.
- On Setup & Onboarding, you explored RetailMart by clicking in pgAdmin.
- Now you can query the METADATA itself – tables about tables.
- That is a superpower.

The psql Shortcuts

[Quick Inspection Commands (psql only)]

```
\dt schema_name.*    -- list all tables in a schema
\d schema.table        -- show columns, types, and constraints
\dn                   -- list all schemas
\l                    -- list all databases
```

Examples:

```
\dt college.*         -- your tables
\d sales.*             -- RetailMart's sales tables
\d sales.orders        -- full column listing for orders
\d customers.customers -- full column listing for customers
```

```
-- These are psql-only shortcuts. In pgAdmin, use the sidebar
-- or right-click → Properties → Columns tab.
```

PRACTICE BLOCK 3 — ALTER TABLE Real Scenarios

Use ALTER TABLE on the tables you created earlier. Do NOT drop and recreate.

SCENARIO: The college dean returns with 5 quick change requests for your college schema tables. Handle each with a single ALTER TABLE statement.

YOUR TASK:

- a)** : Add a 'gpa' column (NUMERIC 3,2) to college.students
- b)** : Add a 'semester' column (INT, DEFAULT 1) to college.students
- c)** : Rename 'credits' in college.courses to 'credit_hours'
- d)** : Add a 'max_marks' column (INT, DEFAULT 100) to college.courses
- e)** : Verify with \d college.students and \d college.courses — do all changes show?

Answer

✓ ANSWER

```
ALTER TABLE college.students ADD COLUMN gpa NUMERIC(3,2);
ALTER TABLE college.students ADD COLUMN semester INT DEFAULT 1;
ALTER TABLE college.courses RENAME COLUMN credits TO credit_hours;
ALTER TABLE college.courses ADD COLUMN max_marks INT DEFAULT 100;
-- e) \d college.students and \d college.courses show the new columns.
```

SCENARIO: You realize several data type choices in your earlier tables were WRONG. Fix them using ALTER TABLE — do NOT recreate the tables.

YOUR TASK:

- a)** : If you used INT for any phone column, change it to VARCHAR(15). Write the ALTER statement and explain why this matters.
- b)** : If you used INT for salary anywhere, change it to NUMERIC(10,2). What is lost if salary stays as INT?
- c)** : Change the students email column from VARCHAR(150) to VARCHAR(200) — some students have very long institutional emails.
- d)** : Add a created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP to EVERY table that does not have one. Write all the ALTER statements.
- e)** : Add an updated_at TIMESTAMP column to college.students. Why is this useful for auditing?

Answer



```
-- a) phone must be text (keeps +91, leading 0s, separators):
ALTER TABLE college.teachers ALTER COLUMN phone TYPE VARCHAR(15);
-- b) salary as exact money (INT loses paisa):
ALTER TABLE college.teachers ALTER COLUMN monthly_salary TYPE NUMERIC(10,2);
-- c) widen email:
ALTER TABLE college.students ALTER COLUMN email TYPE VARCHAR(200);
-- d) add created_at to every table:
ALTER TABLE college.students ADD COLUMN created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE college.courses ADD COLUMN created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE college.teachers ADD COLUMN created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
-- e) updated_at lets you audit when a row last changed:
ALTER TABLE college.students ADD COLUMN updated_at TIMESTAMP;
```

SCENARIO: The CTO wants you to write a migration script — a single .sql file that performs multiple ALTER TABLE operations. This is how real companies evolve databases.

YOUR TASK:

Open VS Code. Create a file called migration_001.sql. Write these operations:

- 1) : Add 2 new columns to college.students (middle_name VARCHAR(50), blood_group VARCHAR(5))
- 2) : Rename 1 column in college.courses
- 3) : Change the data type of 1 column (e.g., capacity from INT to SMALLINT)
- 4) : Drop 1 unnecessary column from any table
- 5) : Set DEFAULT values on 2 columns
- 6) : Add a created_at TIMESTAMP to 1 table

Execute the entire file in psql: \i /path/to/migration_001.sql

Verify ALL changes with \d commands. This is how Flipkart and Razorpay evolve their databases — one migration file at a time.

Answer



```
-- migration_001.sql
ALTER TABLE college.students ADD COLUMN middle_name VARCHAR(50);
ALTER TABLE college.students ADD COLUMN blood_group VARCHAR(5);
ALTER TABLE college.courses RENAME COLUMN department TO dept_name;
ALTER TABLE college.classrooms ALTER COLUMN capacity TYPE SMALLINT;
ALTER TABLE college.library_books DROP COLUMN added_on;
ALTER TABLE college.students ALTER COLUMN is_active SET DEFAULT TRUE;
ALTER TABLE college.classrooms ALTER COLUMN has_ac SET DEFAULT FALSE;
ALTER TABLE college.teachers ADD COLUMN created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
```

SCENARIO: The VP of Engineering asks: 'What would happen if we needed to add a new column to a table with 150,000 rows in production? What are the risks?' Write a technical analysis.

YOUR TASK:

a) : Research (or think through): What happens when you run ALTER TABLE ADD COLUMN on a table with 150,000 existing rows? Do all rows get the new column? What value do they get?

b) : What if the new column has a DEFAULT value? Does PostgreSQL need to rewrite all 150,000 rows? (Hint: since PostgreSQL 11, ADD COLUMN with DEFAULT is instant — no table rewrite needed. Research why.)

c) : What happens if you ALTER COLUMN TYPE on a table with 150,000 rows? Is it instant or does it rewrite the entire table?

d) : Write a safeguarded migration script that adds 3 columns to a large table, with comments explaining each step and its impact on production.

Hint: ADD COLUMN with DEFAULT is instant in PostgreSQL 11+. ALTER TYPE requires a full table rewrite. This distinction is critical for production deployments.

Answer



- a) ADD COLUMN gives every existing row the new column; with no DEFAULT they get NULL. The column appears for all 150,000 rows.
- b) Since PostgreSQL 11, ADD COLUMN ... DEFAULT is INSTANT - the default is stored as metadata, not written to every row, so no table rewrite.
- c) ALTER COLUMN TYPE usually REWRITES the whole table (and locks it) - slow on 150k+ rows. Avoid at peak; batch or use a new column + backfill.
- d) Safeguarded migration:
BEGIN;
SET lock_timeout = '3s'; -- fail fast instead of blocking writes
ALTER TABLE big ADD COLUMN a INT; -- instant
ALTER TABLE big ADD COLUMN b BOOLEAN DEFAULT FALSE; -- instant (v11+)
ALTER TABLE big ADD COLUMN c TIMESTAMP; -- instant
COMMIT;

THE DESIGN CHALLENGE — Think Like a Senior Engineer

This section tests your ability to make design DECISIONS — not just know syntax. Senior engineers earn more because they make the right calls BEFORE code is written.

The IRCTC Booking System — How Would YOU Design It?

(1/3)

IRCTC handles 2+ million bookings per day during peak season. Every ticket involves:

- A passenger (name, age, gender, ID proof, phone, email)
- A train (number, name, route, departure station, arrival station)
- A booking (PNR, coach, seat number, class, fare, booking date, status)
- A payment (amount, method, transaction ID, status, refund info)
- Waitlist management (current position, probability of confirmation)

Imagine you are building this from scratch. Think about:

The IRCTC Booking System — How Would YOU Design It?

(2/3)

1. : How many SCHEMAS would you create? (passengers, trains, bookings, payments, waitlist?)
2. : The PNR is a 10-digit number like 4521378960. Should it be INT or VARCHAR? (Hint: do you ever do SUM(pnr) or AVG(pnr)?)
3. : Train number is like 12301 (Rajdhani). INT or VARCHAR? (Some trains have letters: 02301A)
4. : Fare is ₹2,450.00. What type? Why not INT?
5. : Booking status cycles through: CNF → WL → RAC → CAN. VARCHAR(10) or something else?
6. : departure_time needs to be precise to the minute. DATE or TIMESTAMP?
7. : A single booking can have 6 passengers (family). One table or two?

The IRCTC Booking System — How Would YOU Design It?

(3/3)

There is no single right answer — but there are WRONG answers. A VARCHAR PNR is right (never do math on it). An INT fare is wrong (₹2,450.50 has paisa). A DATE for departure loses the time. These decisions, made on SQL Intro of the project, determine whether the system works or breaks at scale.

Design Exercise — Your IRCTC Tables

```
-- Write CREATE TABLE for at least these 4 tables:  
-- 1. irctc.passengers  
-- 2. irctc.trains  
-- 3. irctc.bookings  
-- 4. irctc.payments  
  
-- For each column, ask the 10-Second Decision Tree:  
-- Is it an auto-ID? → SERIAL  
-- Is it money? → NUMERIC  
-- Is it a code/identifier? → VARCHAR  
-- Is it a moment? → TIMESTAMP  
-- Is it yes/no? → BOOLEAN  
  
-- Compare with a classmate. Different designs are normal.  
-- Discuss: where do you disagree? Who has the better argument?
```

REAL WORLD

WHY THIS EXERCISE MATTERS

In every Data Analyst and Data Engineer interview, you will face a 'design a database for X' question. Common ones:

- Design a database for an e-commerce platform (Flipkart/Amazon)
- Design a database for a ride-sharing app (Ola/Uber)
- Design a database for a food delivery app (Swiggy/Zomato)
- Design a database for a banking system (PhonePe/Paytm)
- Design a database for a school/university management system

The interviewer is testing: Can you identify entities? Can you choose correct types? Do you understand relationships?
Can you think about scale?

You are building this skill RIGHT NOW. Every CREATE TABLE you write today is interview preparation.

INTERVIEW QUESTION BANK — DDL & Data Types Concepts

Practice answering OUT LOUD. Interviews test how clearly you explain, not just whether you know.

Interview Questions — DDL & Data Types

Q1: 'What is DDL? Give me 3 examples.'

A: 'DDL is Data Definition Language — commands that define or modify database structures. CREATE TABLE builds a new table, ALTER TABLE modifies an existing one, and DROP TABLE permanently destroys it. DDL changes the BLUEPRINT, not the data inside it.'

Q2: 'What is the difference between DROP TABLE and DELETE FROM?'

A: 'DROP is DDL — it destroys the entire table including structure and data. The table ceases to exist. DELETE is DML — it removes specific rows but the table structure remains intact. You can INSERT new rows after DELETE. After DROP, there is no table to INSERT into.'

Q3: 'Explain CHAR vs VARCHAR vs TEXT.'

A: 'CHAR(n) is fixed-length — always pads to n characters, wasting storage. Rarely used today. VARCHAR(n) is variable-length — stores only what you give it, up to n characters. Most common for names, emails, codes. TEXT is unlimited length — no max limit. Perfect for reviews, descriptions, long-form content. VARCHAR and TEXT have identical performance in PostgreSQL.'

DDL & Data Types Self-Check (15 Questions)

Close your notes. Answer from memory:

- 1. What does DDL stand for? Name 4 DDL commands.
- 2. Write the CREATE TABLE syntax from memory (template with 3 columns).
- 3. What is the difference between DROP TABLE, TRUNCATE, and DELETE?
- 4. What does SERIAL do? What happens to the ID when you delete a row?
- 5. NUMERIC(10,2) — what does 10 mean? What does 2 mean? Max value?
- 6. Why should phone numbers be VARCHAR, not INT? Give 2 reasons.
- 7. What happens if you store ₹1,299.50 in an INT column?
- 8. What happens if you store money as REAL/FLOAT? Show the math.
- 9. CHAR(50) vs VARCHAR(50) — which is better for names? Why?
- 10. When do you use DATE vs TIMESTAMP? Give one example of each.
- 11. Write an ALTER TABLE command that adds a column with a default.
- 12. Write an ALTER TABLE command that changes a column's data type.
- 13. What is CASCADE in DROP SCHEMA?
- 14. What is IF NOT EXISTS and why should you always use it in scripts?
- 15. Name the 8 most common data types you will use in this course.

Key Takeaways (1/2)

- 1. DDL = Structure:** CREATE builds new objects. ALTER modifies existing ones. DROP destroys permanently. DDL defines the blueprint — what tables exist, what columns, what types. It does not touch data rows.
- 2. CREATE TABLE:** Every column needs a name + data type. Use IF NOT EXISTS for safety. Always specify the schema. Follow snake_case naming conventions.
- 3. Data Types Are Defense:** The right type REJECTS bad data at INSERT time. The wrong type lets it in silently — and you pay the price months later when queries break.

Key Takeaways (2/2)

- 4. The Big Rules:** NUMERIC for money (never FLOAT). VARCHAR for phones/pins (never INT). DATE/TIMESTAMP for dates (never VARCHAR). BOOLEAN for yes/no (never VARCHAR). SERIAL for auto-IDs.
- 5. ALTER TABLE:** Tables evolve constantly. ADD COLUMN, DROP COLUMN, RENAME, ALTER TYPE. Never drop-and-recreate in production — ALTER preserves existing data.
- 6. DROP vs TRUNCATE vs DELETE:** DROP = demolish the room. TRUNCATE = empty the room. DELETE = remove specific furniture. Only DELETE allows WHERE conditions.
- 7. Production Mindset:** DROP is permanent. Always use IF EXISTS. Practice in accio_26. Never touch RetailMart structure. Migration scripts are how real companies manage DDL changes.

DDL & Data Types — DDL & Data Types Quick Reference

-- DDL Commands

```
CREATE DATABASE db_name;
CREATE SCHEMA schema_name;
CREATE TABLE IF NOT EXISTS schema.table (col TYPE, ...);
DROP TABLE IF EXISTS schema.table;
DROP SCHEMA IF EXISTS schema CASCADE;
ALTER TABLE schema.table ADD COLUMN col TYPE DEFAULT val;
ALTER TABLE schema.table DROP COLUMN col;
ALTER TABLE schema.table RENAME COLUMN old TO new;
ALTER TABLE schema.table ALTER COLUMN col TYPE new_type;
ALTER TABLE schema.table ALTER COLUMN col SET DEFAULT val;
ALTER TABLE schema.table RENAME TO new_name;
```

-- Numeric Types

```
SMALLINT      – small integers (-32K to 32K)
INT/INTEGER   – standard integers (±2.1B)
BIGINT        – huge integers (±9.2 quintillion)
SERIAL        – auto-increment INT (1, 2, 3...)
BIGSERIAL     – auto-increment BIGINT
NUMERIC(p,s)  – exact decimal (ALWAYS for money)
```

-- Text Types

```
VARCHAR(n)    – variable length, up to n chars (most common)
TEXT          – unlimited length (reviews, descriptions)
CHAR(n)       – fixed length, padded with spaces (avoid)
```

DDL & Data Types — DDL & Data Types Quick Reference

-- Date/Time Types

DATE – calendar date only (2025-05-16)
TIMESTAMP – date + time (2025-05-16 14:30:00)
TIMESTAMPZ – date + time + timezone
TIME – clock time only (14:30:00)
INTERVAL – duration (3 days 4 hours)

-- Other Types

BOOLEAN – true / false / NULL
UUID – universally unique identifier
JSONB – binary JSON (JSON)
ARRAY – list of values in one cell

-- Golden Rules

Money → NUMERIC(p,s) – never FLOAT/REAL
Phone/Pin → VARCHAR(n) – never INT (leading zeros!)
Names → VARCHAR(n) – never CHAR (wastes space)
Dates → DATE or TIMESTAMP – never VARCHAR
Yes/No → BOOLEAN – never VARCHAR
Long text → TEXT – never VARCHAR(9999)
Auto-ID → SERIAL – never manually manage IDs

DDL & Data Types — DDL & Data Types Quick Reference

-- DROP vs TRUNCATE vs DELETE

DROP TABLE	– destroys structure + data (DDL, permanent)
TRUNCATE	– removes ALL rows, keeps structure (DDL)
DELETE FROM	– removes specific rows via WHERE (DML)

CHALLENGE

CHALLENGE (1/2)

DDL & Data Types Take-Home Challenge

Complete these tasks BEFORE Constraints & DML:

CHALLENGE (2/2)

- 1. Build 3 Tables :** In `accio_26`, create a mini e-commerce schema with: products (8+ columns), customers (7+ columns), orders (8+ columns). Use correct data types for EVERY column. Add DEFAULTs where appropriate.
- 2. ALTER Practice :** Write 6 ALTER TABLE statements that modify your tables — add 2 columns, rename 1, change type of 1, drop 1, set default on 1. Save as a .sql file.
- 3. Type Audit :** Examine 3 RetailMart V3 tables using \d. For each table, write every column name + type. For 5 columns, explain why that type was chosen over alternatives.
- 4. Bug Hunt :** Write a CREATE TABLE with intentionally WRONG types for 5 columns. Swap with a classmate and fix each other's mistakes.
- 5. Quick Quiz :** Without notes, write the correct type for: `customer_id`, `email`, `salary`, `order_date`, `is_active`, `review_text`, `latitude`, `phone_number`, `aadhaar_number`, `rating`.

We add CONSTRAINTS (PK, FK, NOT NULL, CHECK, UNIQUE, DEFAULT) and start DML (INSERT, UPDATE, DELETE)!

YOUR SQL JOURNEY



SQL Intro



Installation



DDL + Types ← YOU ARE HERE

← HERE



Constraints + DML



Filtering (first DQL!)



Joins



CTEs



Window Functions



Indexing



Views



Cohort/RFM



Capstone



WhatsApp Announcement

Tomorrow we add RULES to your tables and start putting DATA in them:

- PRIMARY KEY — uniquely identify every row (no duplicates, no NULLs)
- FOREIGN KEY — link tables together (the 'relational' part!)
- NOT NULL, UNIQUE, CHECK, DEFAULT — data integrity constraints
- INSERT INTO — add rows to your tables
- UPDATE — modify existing data
- DELETE — remove specific rows
- TRUNCATE — clear all rows at once

Today you built the blueprint. Tomorrow you enforce rules and fill it with data.

— See you on Constraints & DML! Siraj Sir